



CS2009

DESIGN AND ANALYSIS OF ALGORITHMS

Dr. Hashim Yasin

**National University of Computer
and Emerging Sciences,
Faisalabad, Pakistan.**

SORTING



Algorithm

3

- An algorithm is a well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**.

For Example:

Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Sorting

4

Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

For example,

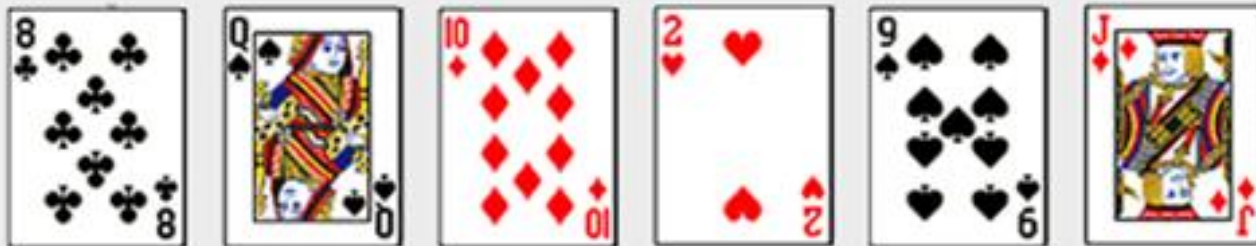
- ▣ given the input sequence (31; 41; 59; 26; 41; 58),
- ▣ a sorting algorithm
- ▣ returns as output sequence (26; 31; 41; 41; 58; 59).
- Such an input sequence is called an ***instance*** of the sorting problem

INSERTION SORT



Insertion Sort

6



Insertion Sort

7

5	1	6	2	4	3
---	---	---	---	---	---

Lets take this Array.



(Always we start with the second element as key.)

As we can see here, in insertion sort, we pick up a key, and compares it with elemnts ahead of it, and puts the key in the right place

5 has nothing before it.

1 is compared to 5 and is inserted before 5.

6 is greater than 5 and 1.

2 is smaller than 6 and 5, but greater than 1, so its is inserted after 1.

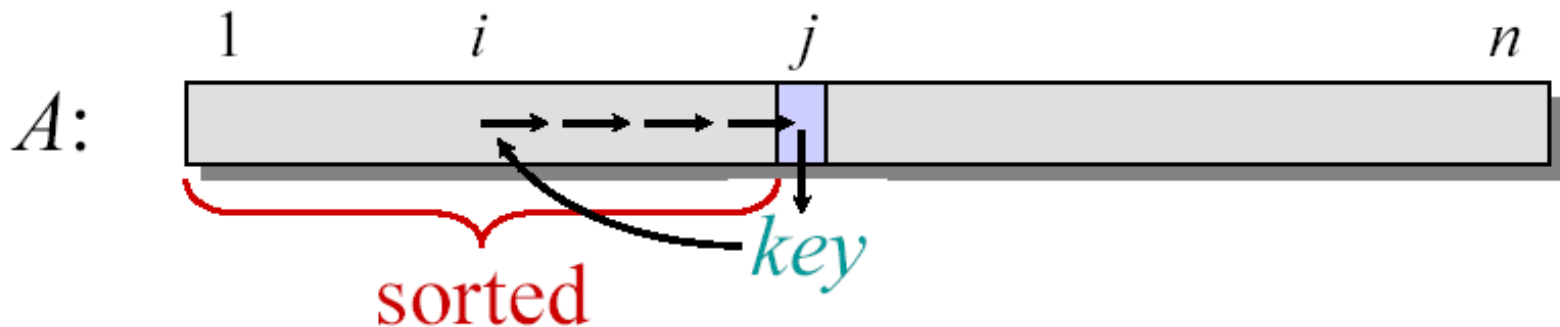
And this goes on...

Insertion Sort

8

“pseudocode”

```
INSERTION-SORT ( $A, n$ )     $\triangleright A[1 \dots n]$   
  for  $j \leftarrow 2$  to  $n$   
    do  $key \leftarrow A[j]$   
       $i \leftarrow j - 1$   
      while  $i > 0$  and  $A[i] > key$   
        do  $A[i+1] \leftarrow A[i]$   
           $i \leftarrow i - 1$   
       $A[i+1] = key$ 
```



Insertion Sort

9

1	2	3	4	5	6
5	3	6	1	4	2

 INSERTION-SORT (A, n) $\triangleright A[1 \dots n]$
 for $j \leftarrow 2$ **to** n
 do $key \leftarrow A[j]$
 $i \leftarrow j - 1$
 while $i > 0$ and $A[i] > key$
 do $A[i+1] \leftarrow A[i]$
 $i \leftarrow i - 1$
 $A[i+1] = key$

1	2	3	4	5	6
5	3	6	1	4	2

Insertion Sort

10

1	2	3	4	5	6
5	3	6	1	4	2

$\text{INSERTION-SORT}(A, n) \quad \triangleright A[1 \dots n]$
for $j \leftarrow 2$ **to** n
 do $key \leftarrow A[j]$
 $i \leftarrow j - 1$
 while $i > 0$ and $A[i] > key$
 do $A[i+1] \leftarrow A[i]$
 $i \leftarrow i - 1$
 $A[i+1] = key$

$j = 4$ ←
 $Key = 1$ ←
 $i = 3$ ←
 $0 > 0$ and $3 > 1$ ←
 False
 $A[1] \leftarrow 1$ ←

1	2	3	4	5	6
1	3	5	6	4	2

Insertion Sort

11

□ Worst Case:

5	4	3	2	1
---	---	---	---	---

□ Best Case:

1	2	3	4	5
---	---	---	---	---

Insertion Sort

12

INSERTION-SORT(<i>A</i>)	<i>cost</i>	<i>times</i>
1 for <i>j</i> = 2 to <i>A.length</i>	c_1	n
2 $key = A[j]$	c_2	$n - 1$
3 // Insert $A[j]$ into the sorted sequence $A[1 \dots j - 1]$.	0	$n - 1$
4 $i = j - 1$	c_4	$n - 1$
5 while $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^n t_j$
6 $A[i + 1] = A[i]$	c_6	$\sum_{j=2}^n (t_j - 1)$
7 $i = i - 1$	c_7	$\sum_{j=2}^n (t_j - 1)$
8 $A[i + 1] = key$	c_8	$n - 1$

$$\begin{aligned} T(n) = & c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\ & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1) . \end{aligned}$$

Insertion Sort (Best Case)

13

INSERTION-SORT(*A*)

1 **for** *j* = 2 **to** *A.length*

2 $key = A[j]$

3 // Insert $A[j]$ into the sorted
 sequence $A[1..j-1]$.

4 $i = j - 1$

5 **while** $i > 0$ and $A[i] > key$

6 $A[i+1] = A[i]$

7 $i = i - 1$

8 $A[i+1] = key$

cost

times

c_1

n

c_2

$n - 1$

0

$n - 1$

c_4

$n - 1$

c_5

$\sum_{j=2}^n t_j$

c_6

$\sum_{j=2}^n (t_j - 1)$

c_7

$\sum_{j=2}^n (t_j - 1)$

c_8

$n - 1$

$$T(n) = c_1 n + c_2 (n - 1) + c_4 (n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8 (n - 1).$$

$c_5(n - 1)$

Insertion Sort (Best Case)

14

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + \cancel{c_5 \sum_{j=2}^n t_j} + c_6 \cancel{\sum_{j=2}^n (t_j - 1)} + c_7 \cancel{\sum_{j=2}^n (t_j - 1)} + c_8(n-1) .$$

$c_5(n-1)$

Best Case:

1	2	3	4	5
---	---	---	---	---

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

It is thus a linear function of n

$$T(n) = O(n)$$

Insertion Sort (Worst Case)

15

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1).$$

Worst Case:

5	4	3	2	1
---	---	---	---	---

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1)$$

$$= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n - (c_2 + c_4 + c_5 + c_8).$$

Insertion Sort (Worst Case)

16

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\ + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1).$$

Worst Case:

5	4	3	2	1
---	---	---	---	---

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

and

$$\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$$

$n = 5$

$$\sum_{j=2}^n j = 2 + 3 + 4 + 5 = 14$$

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1 = \frac{5(5+1)}{2} - 1 = 14$$

Insertion Sort (Worst Case)

17

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1).$$

Worst Case:

5	4	3	2	1
---	---	---	---	---

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) \\ &\quad + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\ &\quad - (c_2 + c_4 + c_5 + c_8). \end{aligned}$$

$T(n) = O(n^2)$

Insertion Sort

18

□ Best case:

elements already sorted, $t_i = 1$,
running time = $f(n)$, i.e., *linear time*.

□ Worst case:

elements are sorted in inverse order,
 $t_i = i$, running time = $f(n^2)$, i.e., *quadratic time*

□ Average case:

$t_i = i/2$, running time = $f(n^2)$,
i.e., *quadratic time*

Review on Summations

```
MaxSubvector(A, n)
    maxsum ← 0;
    for i ← 1 to n
        for j = i to n
            sum ← 0
            for k ← i to j
                sum += A[k]
            maxsum ← max(sum, maxsum)
    return maxsum
```

$$T(n) = \sum_{i=1}^n \sum_{j=i}^n \sum_{k=i}^j 1$$

NOTE: This is not a simplified solution. What is the final answer?

Review on Summations

- **Constant Series:** For integers a and b , $a \leq b$,

$$\sum_{i=a}^b 1 = b - a + 1$$

- **Linear Series (Arithmetic Series):** For $n \geq 0$,

$$\sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

- **Quadratic Series:** For $n \geq 0$,

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Review on Summations

- **Cubic Series:** For $n \geq 0$,

$$\sum_{i=1}^n i^3 = 1^3 + 2^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

- **Geometric Series:** For real $x \neq 1$,

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1} - 1}{x - 1}$$

Review on Summations

□ **Linear-Geometric Series:** For $n \geq 0$, real $c \neq 1$,

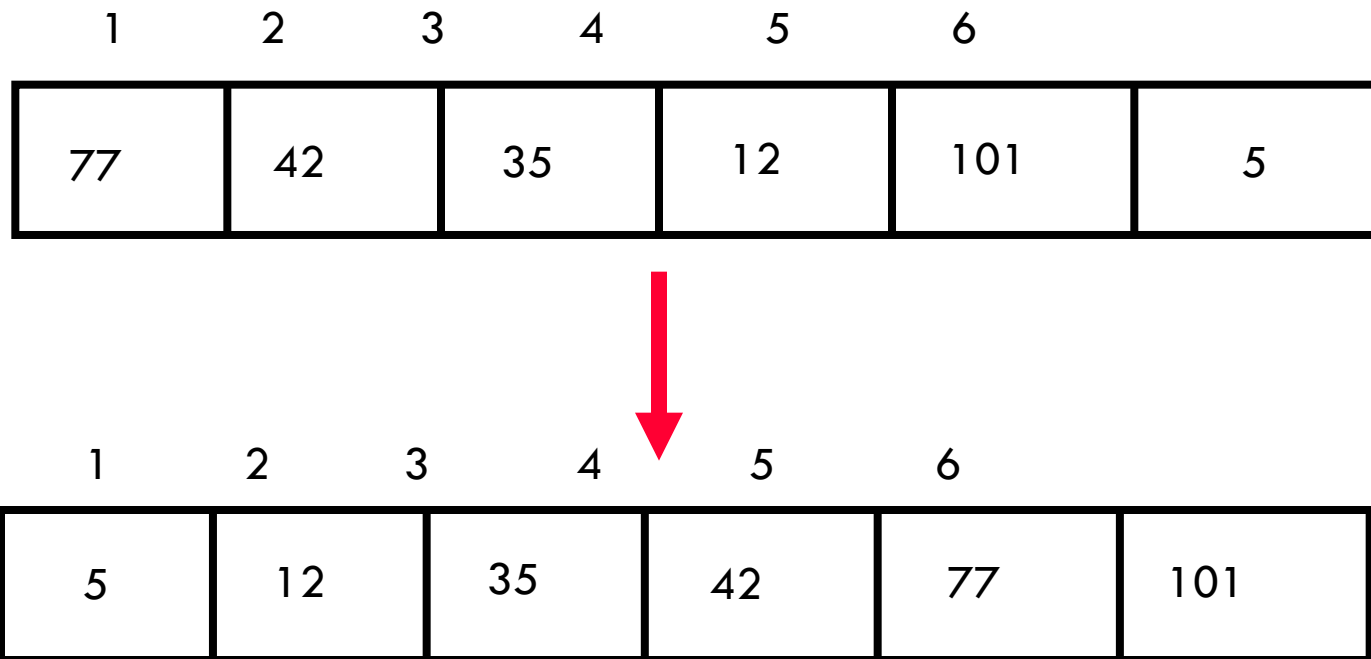
$$\sum_{i=1}^n ic^i = c + 2c^2 + \cdots + nc^n = \frac{-(n+1)c^{n+1} + nc^{n+2} + c}{(c-1)^2}$$

BUBBLE SORT



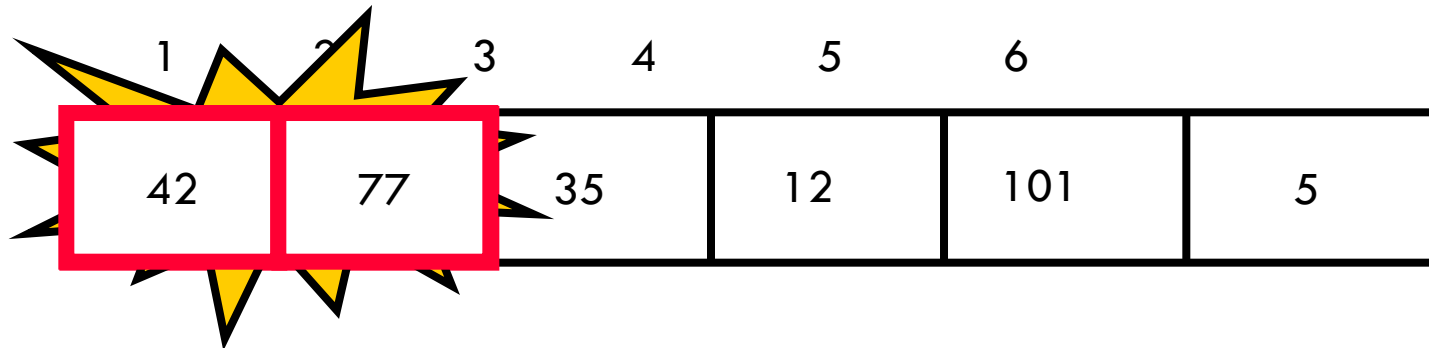
Bubble Sort

- **Sorting takes an unordered collection and makes it an ordered one.**



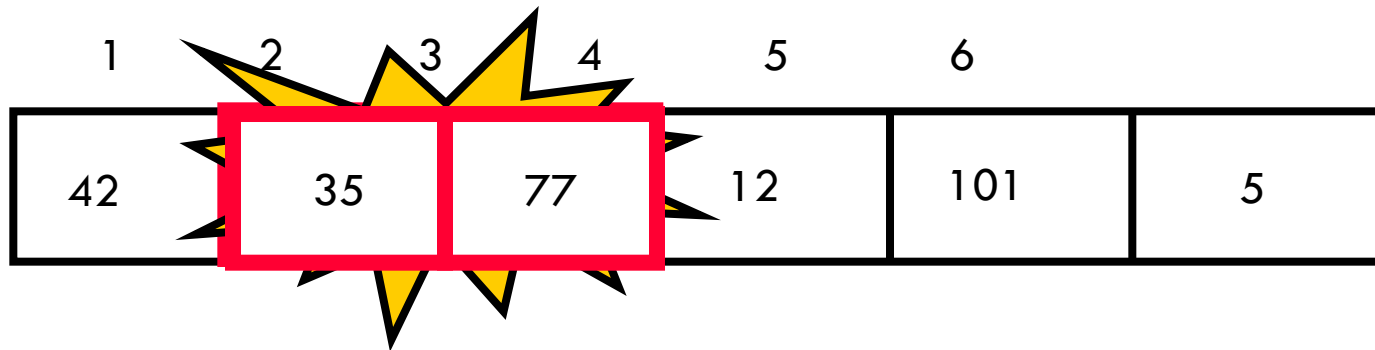
"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - ▣ Move from the front to the end
 - ▣ “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**



"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - ▣ Move from the front to the end
 - ▣ “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**



"Bubbling Up" the Largest Element

- **Traverse a collection of elements**

- ▣ Move from the front to the end

- ▣ “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**

1	2	3	4	5	6
42	35	12	77	5	101

Largest value correctly placed

Items of Interest

- Notice that only the largest value is correctly placed
- All other values are still out of order
- So, we need to repeat this process

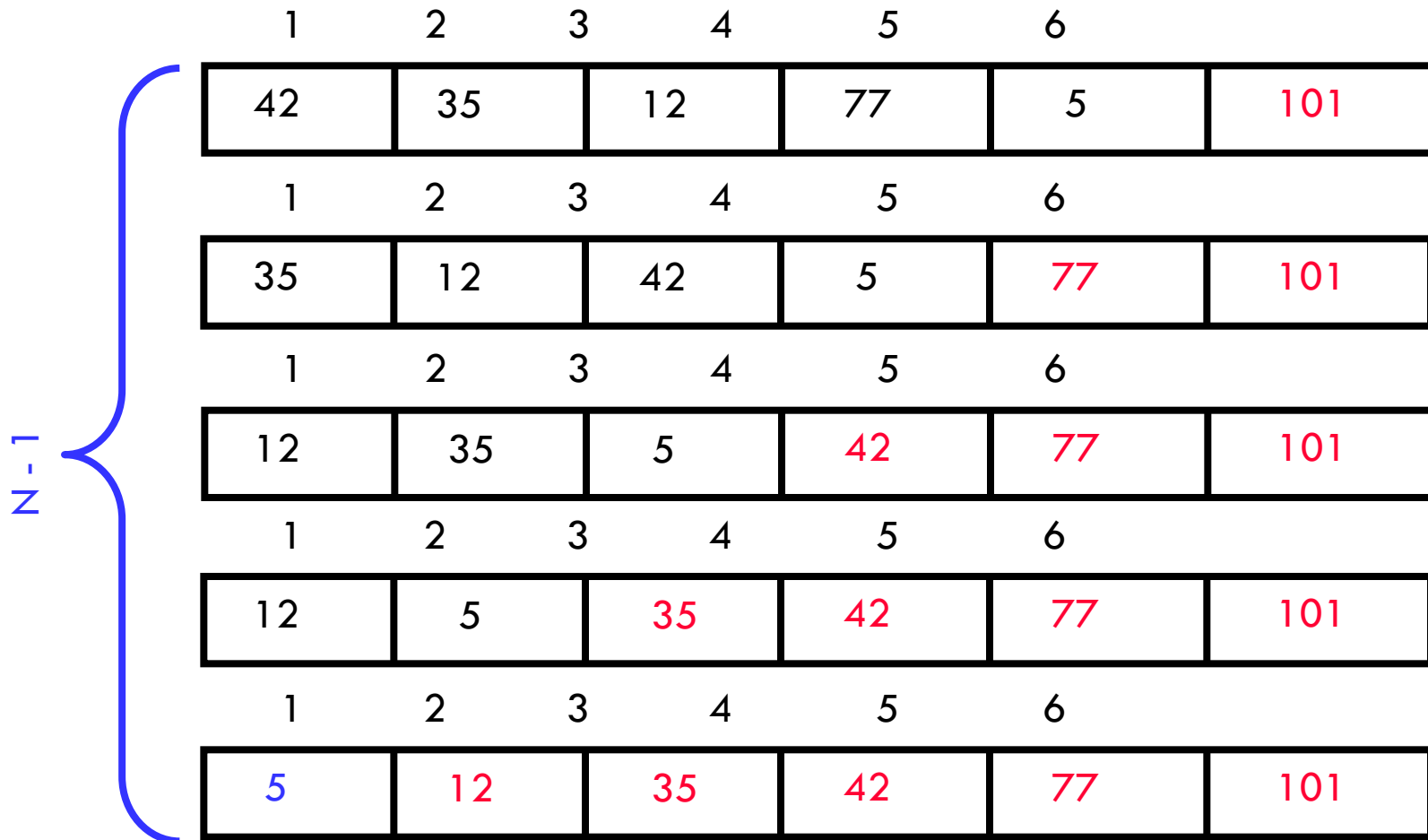
1	2	3	4	5	6
42	35	12	77	5	101

Largest value correctly placed

Repeat “Bubble Up”

- If we have N elements...
- And if each time we bubble an element, we place it in its correct location...
- Then we repeat the “bubble up” process $N - 1$ times.
- This guarantees we'll correctly place all N elements.

“Bubbling” All the Elements

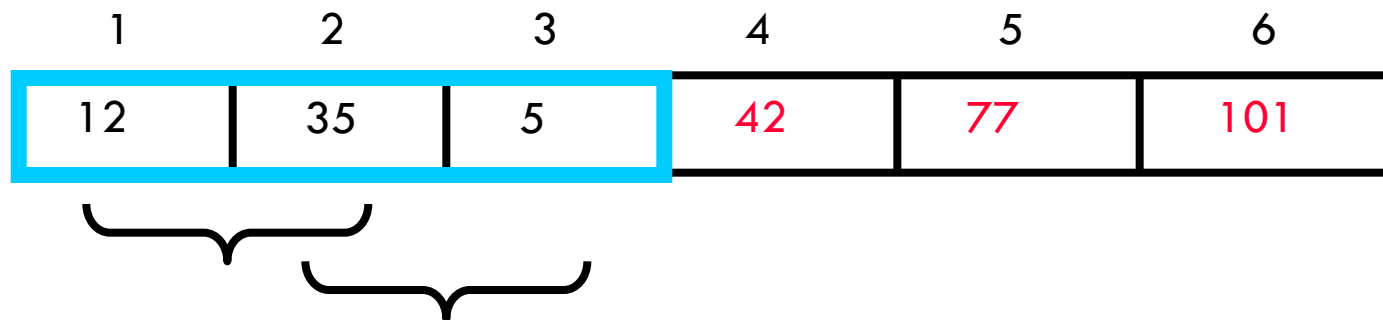


Reducing the Number of Comparisons

1	2	3	4	5	6
77	42	35	12	101	5
1	2	3	4	5	6
42	35	12	77	5	101
1	2	3	4	5	6
35	12	42	5	77	101
1	2	3	4	5	6
12	35	5	42	77	101
1	2	3	4	5	6
12	5	35	42	77	101

Reducing the Number of Comparisons

- On the N^{th} “bubble up”, we only need to do **MAX-N comparisons**.
- For example:
 - ▣ This is the 4th “bubble up”
 - ▣ MAX is 6
 - ▣ Thus, we have **2 comparisons** to do



Standard (Unoptimized) Bubble Sort

```
int n = N; // N is the size of the array;
for (int i = 0; i < N; i++){
    for (int j = 1; j < n; j++) {
        if (A[j] < A[j-1]) {
            swap(j-1, j);
        } //end if
    } //end inner for
    n = n-1;
} //end outer for
```

```
// Swap function assumes that
// A[n] is a globally declared array
swap ( x , y ) {
    int temp = A[x];
    A[x] = A[y];
    A[y] = temp;
}
```



Algorithm does not exit
until all the data is checked

Complexity ? $O(N^2)$

Optimized Bubble Sort

```
int n = N; // N is the size of the array;
for (int i = 0; i < N; i++){
    int swapped = 0;
    for (int j = 1; j < n; j++) {
        if (A[j] < A[j-1]) {
            swap(j-1, j);
            swapped = 1;
        } //end if
    } //end inner for
    n = n-1;
    if (swapped == 0)
        break;
} //end outer for
```

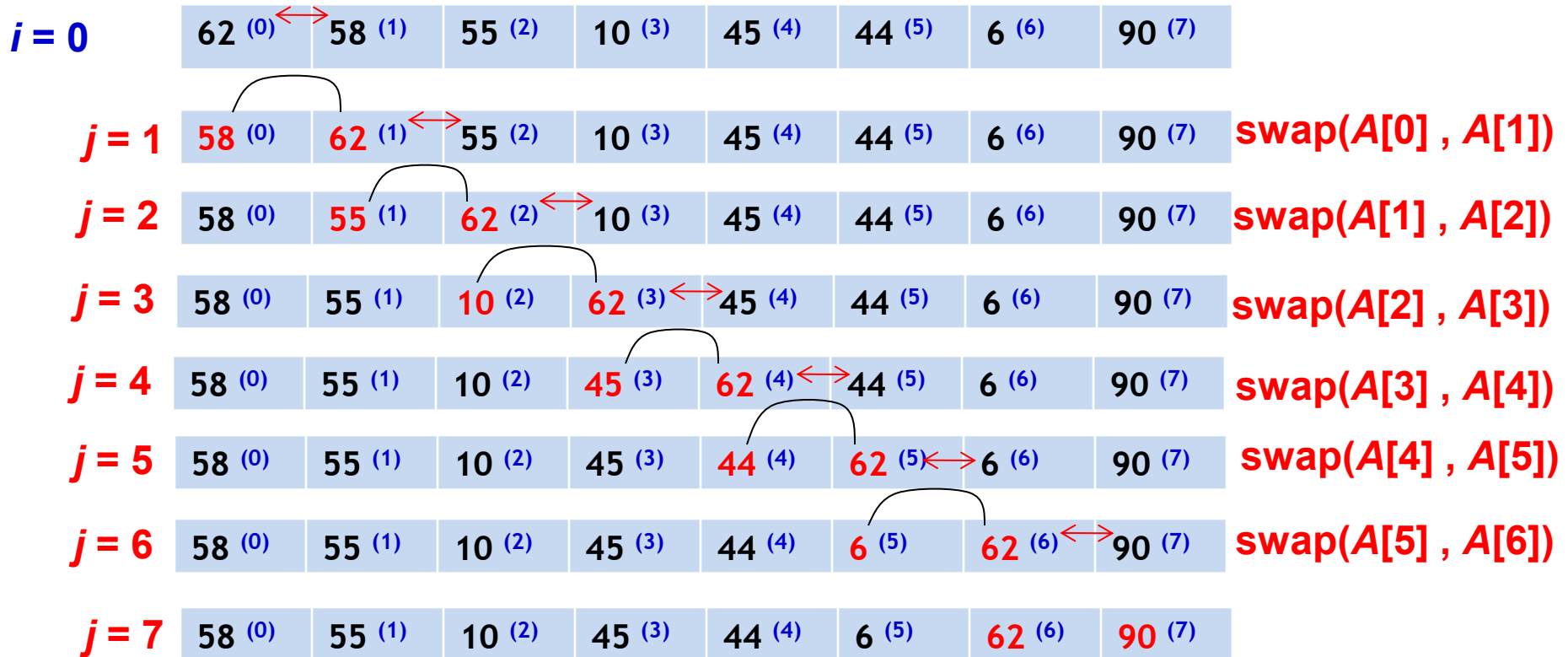
```
// Swap function assumes that
// A[n] is a globally declared array
swap ( x , y) {
    int temp = A[x];
    A[x] = A[y];
    A[y] = temp;
}
```



Algorithm exits if no swap done
in previous (outer loop) step

Complexity ? $O(N^2)$

Bubble Sort Example (First Pass)



```
int n = N; // N is the size of the array;
```

```
for (int i = 0; i < N; i++){
```

```
    int swapped = 0;
```

```
    for (int j = 1; j < n; j++){
```

```
        if (A[j] < A[j-1]) {
```

```
            swap(j-1, j); swapped = 1;
```

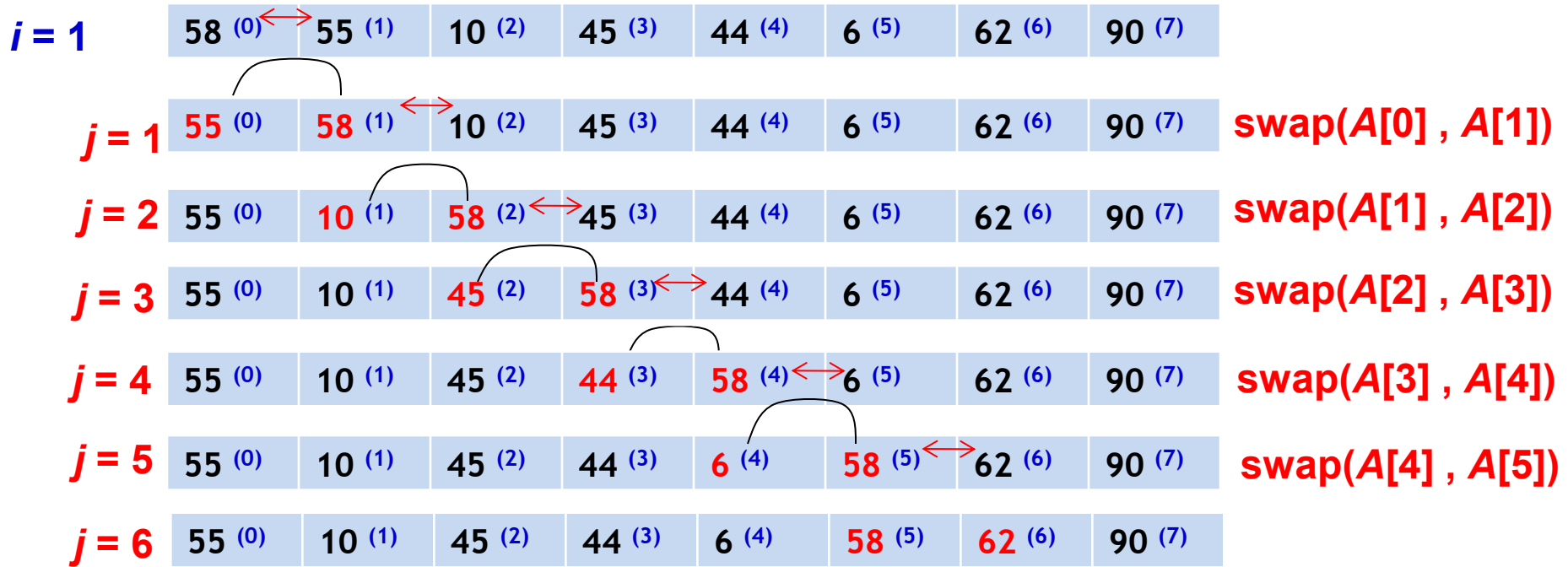
```
        } //end if
```

```
    n = n-1;
```

```
    if (swapped == 0)
```

```
        break; } //End outer for
```

Bubble Sort Example (Second Pass)



```

int n = N; // N is the size of the array;
for (int i = 0; i < N; i++){
    int swapped = 0;
    for (int j = 1; j < n; j++){
        if (A[j] < A[j-1]) {
            swap(j-1 , j); swapped = 1;
        } //end if
    }
    n = n-1;
    if (swapped == 0)
        break; //End outer for
    }

```

Bubble Sort Example (Third Pass)

$i = 2$

55 (0)	10 (1)	45 (2)	44 (3)	6 (4)	58 (5)	62 (6)	90 (7)
--------	--------	--------	--------	-------	--------	--------	--------

$j = 1$

$j = 2$

$j = 3$

$j = 4$

$j = 5$

10 (0)	45 (1)	44 (2)	6 (3)	55 (4)	58 (5)	62 (6)	90 (7)
--------	--------	--------	-------	--------	--------	--------	--------

```
int n=N; // N is the size of the array;
for (int i = 0; i < N; i++){
    int swapped = 0;
    for (int j = 1; j < n; j++){
        if (A[j] < A[j-1]) {
            swap(j-1 , j); swapped = 1;
        } //end if
    }
    n = n-1;
    if (swapped == 0)
        break; } //End outer for
```

Bubble Sort Example (Seventh Pass)

***i* = 6**

6 ⁽⁰⁾	10 ⁽¹⁾	44 ⁽²⁾	45 ⁽³⁾	55 ⁽⁴⁾	58 ⁽⁵⁾	62 ⁽⁶⁾	90 ⁽⁷⁾
------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------

***j* = 1**

6 ⁽⁰⁾	10 ⁽¹⁾	44 ⁽²⁾	45 ⁽³⁾	55 ⁽⁴⁾	58 ⁽⁵⁾	62 ⁽⁶⁾	90 ⁽⁷⁾
------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------

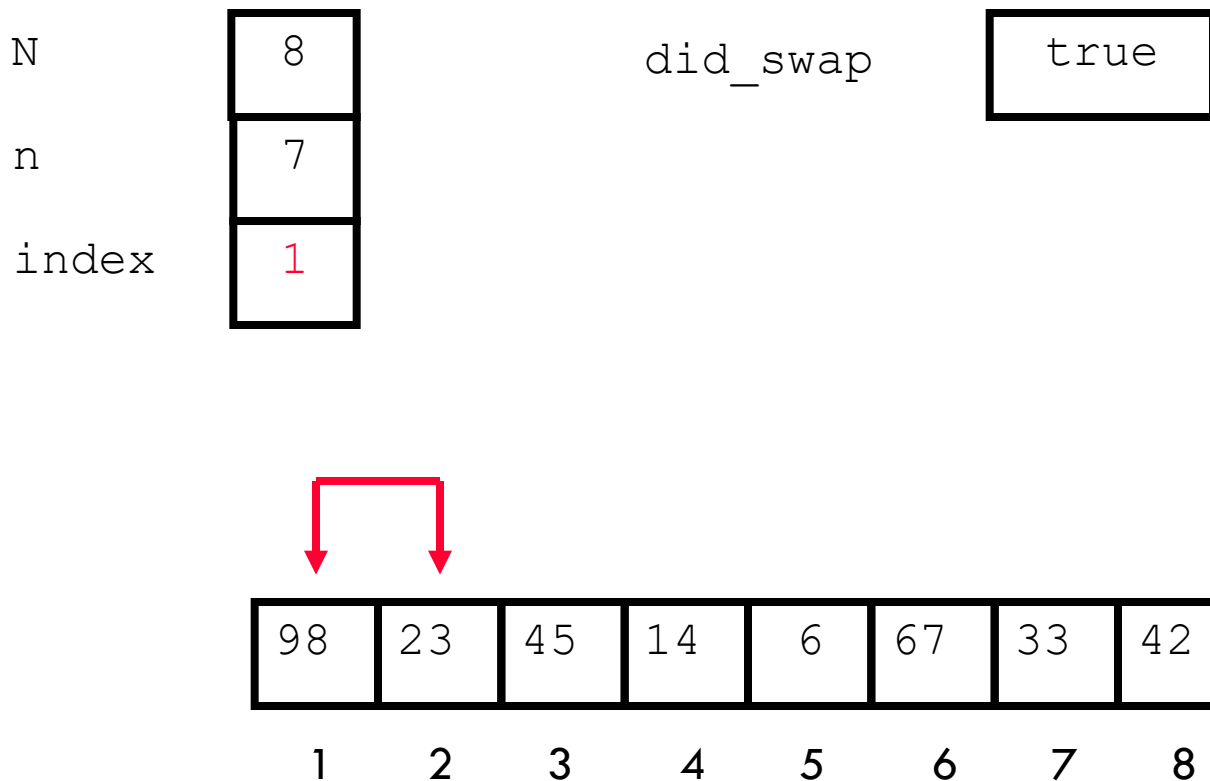
```
int n=N; // N is the size of the array;
for (int i = 0; i < N; i++){
    int swapped = 0;
    for (int j = 1; j < n; j++){
        if (A[j] < A[j-1]) {
            swap(j-1 , j); swapped = 1;
        } //end if
    }
    n = n-1;
    if (swapped == 0)
        break;           } //End outer for
```

EXAMPLE 3



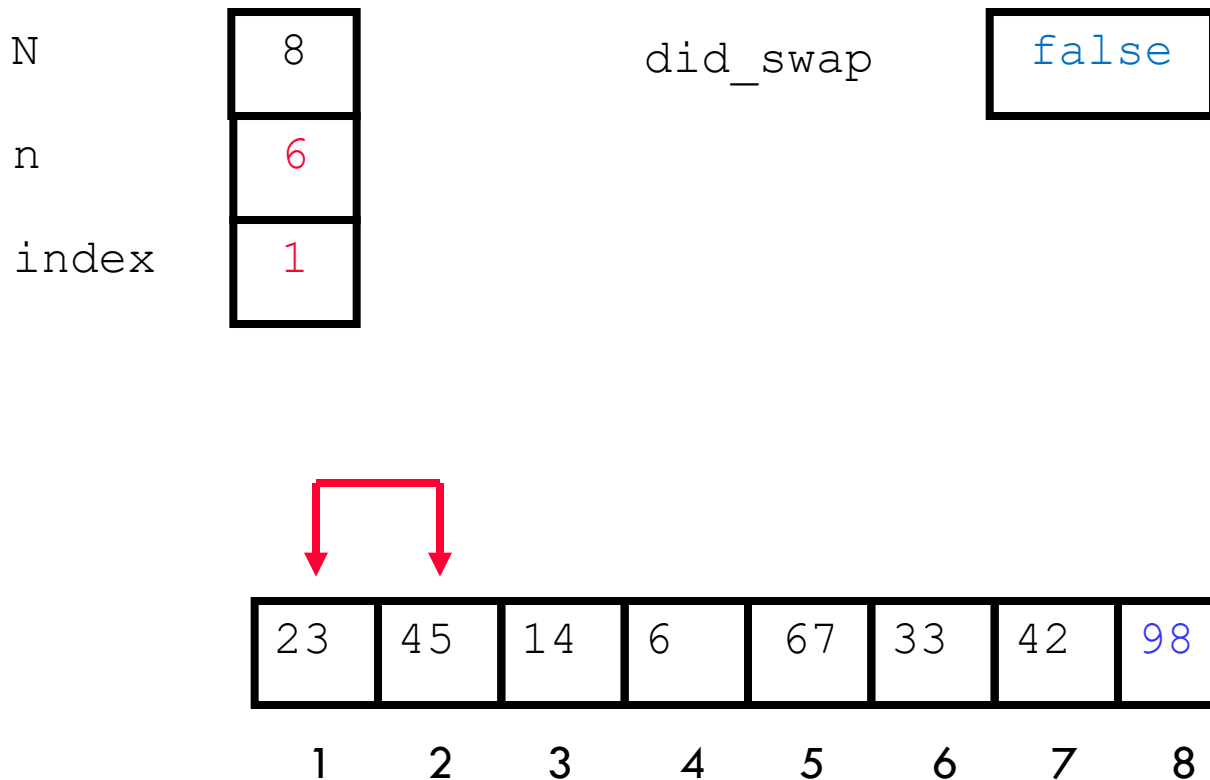
An Animated Example

40



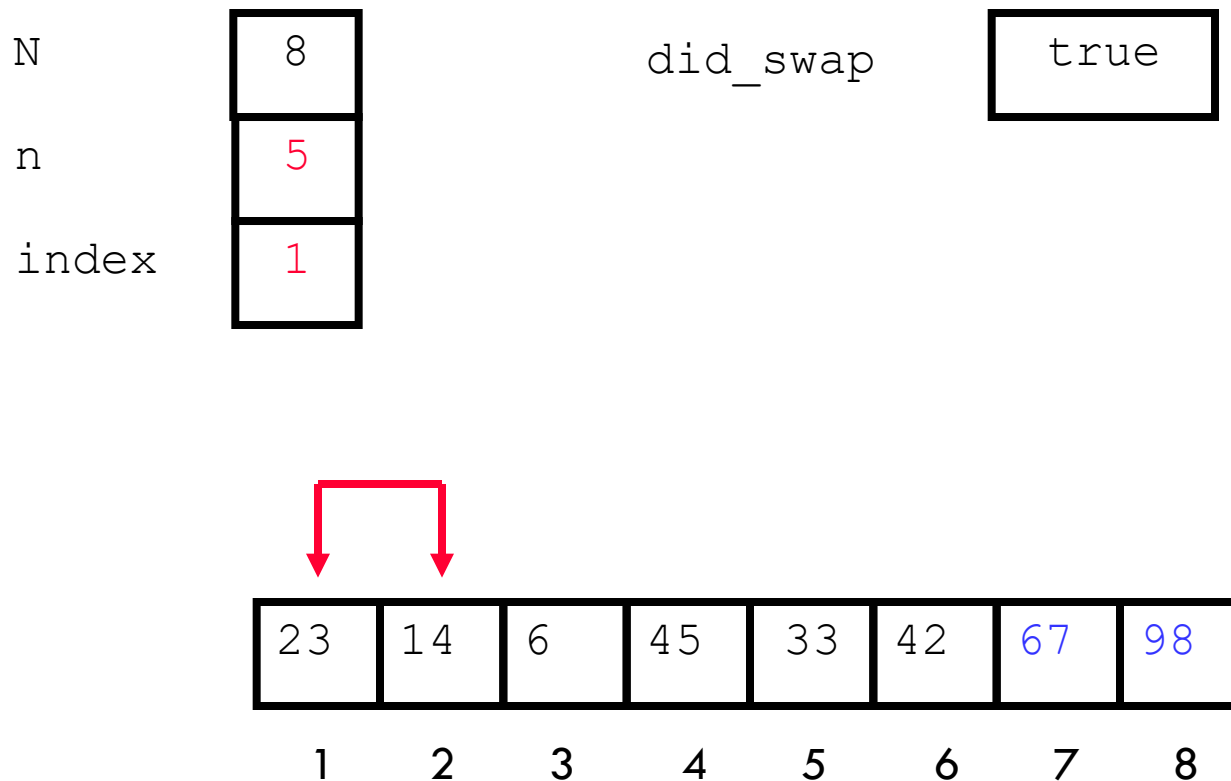
The Second “Bubble Up”

41



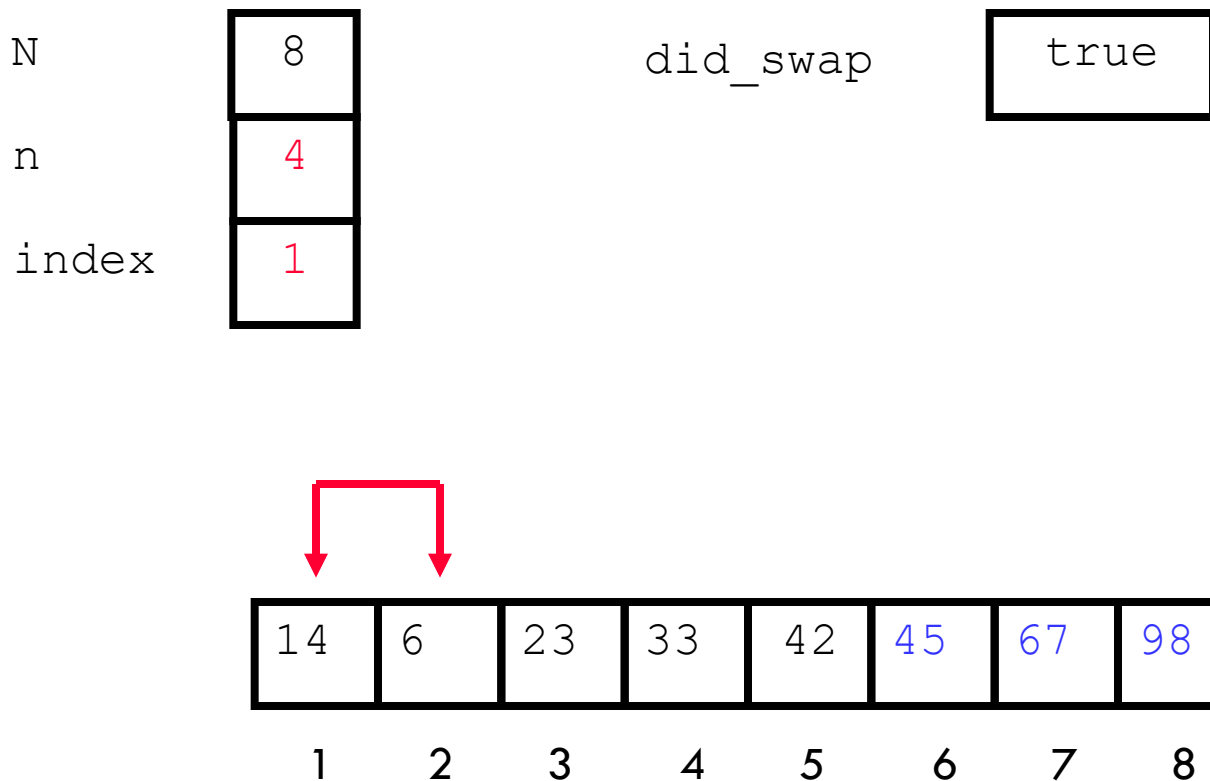
The Third “Bubble Up”

42



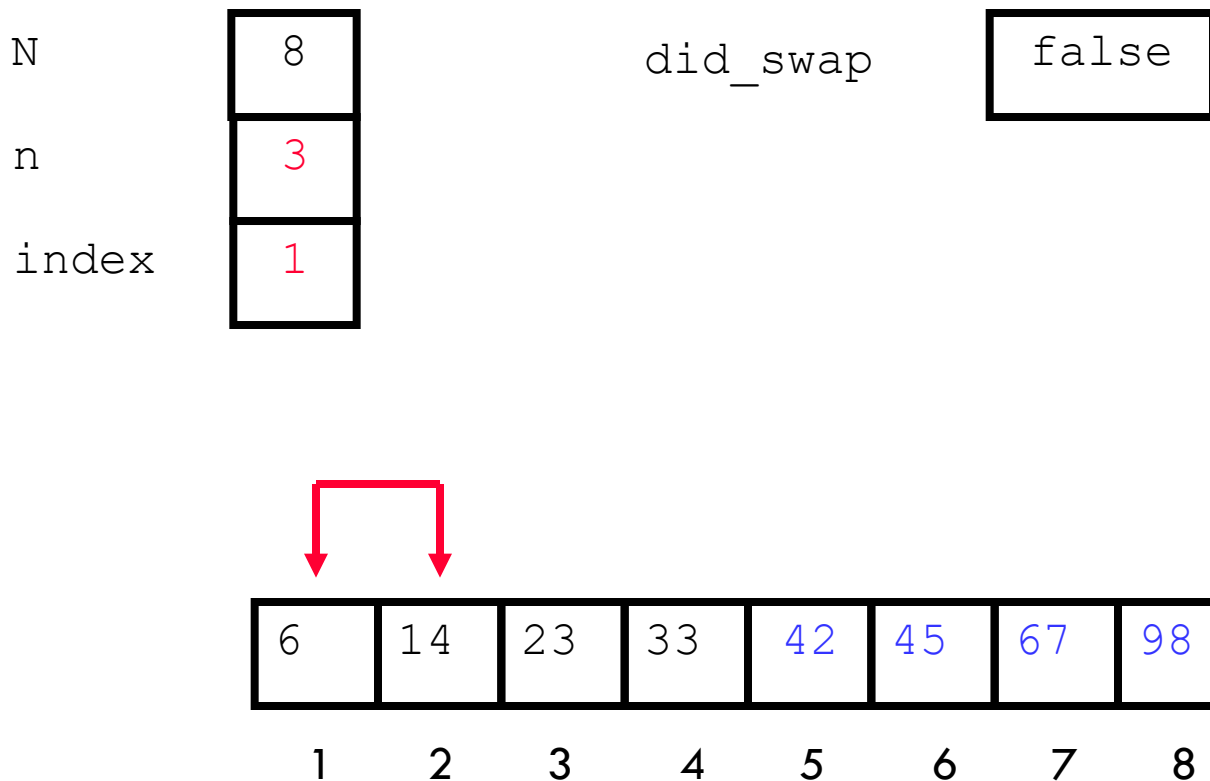
The Fourth “Bubble Up”

43



The Fifth “Bubble Up”

44



Acknowledgement

45

Introduction to Algorithms

Thomas H. Cormen, Charles E. Leiserson,
Ronald L. Rivest, and Clifford Stein